

AI-Driven Multi-Agent Diagnostic and Observability System

This comprehensive report details the architecture, design, and operational workflows for the AI-driven Multi-Agent Diagnostic and Observability System, as synthesized from the provided sources.

1. Architectural Overview and Design Philosophy

The system is engineered on a modular, security-first architecture designed to autonomously correlate signals across telemetry, source code, and operational documentation. It implements a **Multi-Agent Orchestrator** pattern, where a central supervisor manages specialized domain agents. This approach abstracts vendor-specific implementations through standardized protocols like the Model Context Protocol (MCP) while ensuring high-fidelity root cause analysis (RCA).

2. Core Component Specification

Component Category	Details & Responsibilities
A. Supervisor Orchestrator (The Host)	The Supervisor is the central decision engine, utilizing a state-machine framework (e.g., LangGraph) to manage the investigation lifecycle. Unlike linear scripts, it constructs a dynamic investigation graph where execution nodes are modified in real-time based on preceding agent outputs. Responsibilities: Dynamic decomposition of incidents, managing parallel vs. sequential execution, agent invocation, and final evidence synthesis into a diagnostic narrative.
B. Specialized Worker Agents (The Clients)	• Telemetry/RCA Agent: Queries metrics and logs (PromQL, KQL) to detect spikes, saturation, and regional anomalies. • Code/Fix Agent: Uses

Component Category	Details & Responsibilities
	Retrieval-Augmented Generation (RAG) to correlate regressions with specific commits and proposes remediation patches. • Context/Doc Agent: Extracts intent from tickets (Jira), runbooks, and RFCs to differentiate between accidental regressions and intentional changes.
C. Retrieval and Memory Layer	This layer utilizes Milvus or Chroma for semantic storage of code, documentation, and prior diagnostics. It grounds agent responses in actual historical context to eliminate hallucinations.
D. Infrastructure and Security	• Redis Task Queue: Serves as the asynchronous processing layer, decoupling the UI from long-running investigations and providing state checkpointing for fault tolerance. • Security Layer: Performs automated PII masking (scrubbing IPs, tokens, emails) and enforces policy-driven redaction before any data is processed by the LLM. • Hybrid Data Bridge: Facilitates secure on-prem connectivity via multiplexed gRPC streams initiated from the customer environment (Zero-Open-Port architecture).

3. System Diagrams

A. Flow Chart: The Investigation Lifecycle

This chart represents the logical flow of an incident from intake to resolution.

```
graph TD
    A[Issue Intake: User/Alert] --> B[Decomposition: Supervisor  
Graph Generation]
    B --> C{Parallel Retrieval}
```

```

C --> D[Telemetry Agent: Metrics/Logs]
C --> E[Code Agent: Git/RAG]
C --> F[Context Agent: Jira/Docs]
D & E & F --> G[Security Layer: PII Masking/Redaction]
G --> H[Correlation: Evidence Synthesis & Confidence Scoring]
H --> I[Output: UI Presentation via WebSockets]
I --> J{HITL Gate: Human Review}
J -- Rejected --> K[Refine Logic/Adjust Memory]
J -- Approved --> L[Remediation Execution]
L --> M[Update Incident Memory in Vector DB]

```

B. Sequence Diagram: Asynchronous Orchestration

This diagram highlights the interactions between the UI, Redis, and the Supervisor.

```

sequenceDiagram
    participant User
    participant UI
    participant Redis
    participant Supervisor
    participant Agents
    participant MCP

    User->>UI: Submit Incident
    UI->>Redis: Enqueue Investigation Task
    UI-->>User: Return Task ID (Responsive UI)
    Supervisor->>Redis: Poll/Fetch Task
    Note over Supervisor: State-Machine Start
    Supervisor->>Redis: Checkpoint Initial State
    Supervisor->>Agents: Delegate Domain Tasks
    par Parallel Retrieval
        Agents->>MCP: Query Data Sources
        MCP-->>Agents: Raw Data
        Agents->>Agents: Apply PII Masking
        Agents-->>Supervisor: Grounded Insights
    end
    Supervisor->>Redis: Update Status/Checkpoint
    Redis-->>UI: Push Progress via WebSockets
    UI-->>User: Live Progress Updates
    Supervisor->>Supervisor: Evidence Synthesis & Scoring
    Supervisor->>UI: Request HITL Approval

```

C. Class Diagram: System Structure

This represents the structural relationships between the primary architectural entities.

```
classDiagram
    class Supervisor {
        +State Graph state
        +decompose(incident)
        +synthesize(evidence)
        +checkpoint(redis)
    }
    class WorkerAgent {
        +String domain
        +querySource()
        +applyRAG()
    }
    class MCPConnector {
        +String toolType
        +executeRPC()
    }
    class SecurityLayer {
        +maskPII(data)
        +applyPolicy(source)
    }
    class VectorDB {
        +storeEmbedding(context)
        +search(query)
    }
    class RedisQueue {
        +enqueueTask(task)
        +saveState(node)
    }

    Supervisor "1" *-- "many" WorkerAgent : Manages
    WorkerAgent "1" -- "1" SecurityLayer : Passes through
    WorkerAgent "1" -- "1" MCPConnector : Uses
    WorkerAgent "1" -- "1" VectorDB : Retrieves from
    Supervisor "1" -- "1" RedisQueue : Checkpoints to
    MCPConnector "1" -- "many" ExternalSource : Connects
```

4. Operational Scenarios and Scoping

1. **Scenario: Recurring Performance Regression**
 - **Detection:** An alert triggers the Supervisor.

- **Memory Search:** The Context Agent queries the Vector DB and finds a semantically similar incident from the past.
 - **Pattern Match:** The Telemetry Agent confirms the same metric spikes (e.g., p95 latency) identified in the previous incident.
 - **Confidence Tuning:** Because this matches a resolved "Incident Memory" entry, the Supervisor generates a high Confidence Score.
 - **HITL Gate:** The operator sees the historical context via WebSockets and quickly authorizes the known fix.
2. **Scenario: Hybrid Cloud Ingestion**
- **Connection:** On-prem servers initiate an outbound gRPC tunnel to the cloud orchestrator.
 - **Retrieval:** The Code Agent pulls private source code through this tunnel.
 - **Security:** Before the code leaves the customer perimeter (or before LLM processing), the Security Layer redacts internal tokens.
 - **Analysis:** The LLM processes only the neutralized code to identify the root cause without exposing sensitive data.

5. Production Readiness and Scaling Summary

- **Horizontal Scaling:** Worker pools (e.g., Telemetry-heavy vs. Code-heavy) can be scaled independently based on workload demand.
- **Fault Tolerance:** Automated retries and orchestration checkpointing ensure that transient connector failures do not restart the entire investigation.
- **Continuous Improvement:** The system uses a feedback loop where human acceptance or rejection of remediation suggestions tunes the Confidence Scoring algorithm for future incidents.